

Computação de Alto Desempenho

Trabalho 1

Daniel Rebouças de Sousa Barros

DRE: 1232173542

Junho de 2026

Exercício 1 - Variantes do Hello World

(a) Introspecção em nível de SO.

Para esse item, utilizamos o código disponível no Classroom como base e fazemos pequenas modificações. Duas novas dependências devem ser incluídas: `unistd.h` (para usar `getpid()`) e `sched.h` (para usar `sched_getcpu()`).

Rodando `mpi_hello` várias vezes, temos:

```
1 $ mpiexec --oversubscribe -n 8 ./mpi_hello
2 Hello from rank 0/8 -- PID = 9304, CPU = 0
3 Hello from rank 1/8 -- PID = 9305, CPU = 8
4 Hello from rank 2/8 -- PID = 9306, CPU = 11
5 Hello from rank 3/8 -- PID = 9307, CPU = 9
6 Hello from rank 4/8 -- PID = 9308, CPU = 4
7 Hello from rank 5/8 -- PID = 9309, CPU = 10
8 Hello from rank 6/8 -- PID = 9310, CPU = 10
9 Hello from rank 7/8 -- PID = 9311, CPU = 5
10 $ mpiexec --oversubscribe -n 8 ./mpi_hello
11 Hello from rank 0/8 -- PID = 9341, CPU = 0
12 Hello from rank 1/8 -- PID = 9342, CPU = 7
13 Hello from rank 2/8 -- PID = 9343, CPU = 3
14 Hello from rank 3/8 -- PID = 9344, CPU = 10
15 Hello from rank 4/8 -- PID = 9345, CPU = 1
16 Hello from rank 5/8 -- PID = 9346, CPU = 8
17 Hello from rank 6/8 -- PID = 9347, CPU = 13
18 Hello from rank 7/8 -- PID = 9348, CPU = 9
```

Podemos observar, como o esperado, que os PIDs são diferentes e que as CPUs alocadas mudam a cada execução do comando.

OBS.: precisei rodar com essa flag `--oversubscribe` para conseguir usar $n = 8$. Sem isso, dava um erro.

(b) Variante em anel

```
1 $ mpiexec --oversubscribe -n 8 ./hello_ring
2 Hello from rank 0/8 -- PID = 21422, CPU = 4
3 Hello from rank 7/8 -- PID = 21429, CPU = 0
4 Hello from rank 6/8 -- PID = 21428, CPU = 12
5 Hello from rank 5/8 -- PID = 21427, CPU = 2
6 Hello from rank 4/8 -- PID = 21426, CPU = 9
7 Hello from rank 3/8 -- PID = 21425, CPU = 13
8 Hello from rank 2/8 -- PID = 21424, CPU = 8
9 Hello from rank 1/8 -- PID = 21423, CPU = 6
```

A proposta do trabalho pede para justificar a escolha de método para prevenir deadlocks que poderiam ocorrer usando Recv-antes-de-Send. Particularmente, optei por usar o `MPI_Sendrecv`, que aciona buffers separados e garante que as operações de envio e recebimento ocorram simultaneamente pela biblioteca, evitando o bloqueio cíclico (onde todo processo ficaria preso em um `MPI_Recv` aguardando infinitamente o vizinho).

De forma mais pessoal, optei por utilizar um novo comando MPI para me familiarizar mais com a ferramenta.

Exercício 2 - Regra do Trapézio, Generalizada

(a) Balanceamento de carga para n arbitrário.

Nesse item, devemos calcular o valor aproximado de

$$\int_0^{\pi} \sin(x) dx$$

```
1 $ mpiexec -n 3 ./mpi_trap_generalized
2 0 3.141592653589793 10
3 Enter a, b, n: With n = 10 trapezoids, integral of f on [0, 3.14159] =
   1.983523537509454e+00
```

Essa é a aproximação dada pelo programa usando 3 processos e 10 trapezoides. O valor exato dessa integral é 2.

(b) Estudo de escalabilidade forte

```
1 $ mpiexec -n 1 ./mpi_trap_generalized
2 0 3.141592653589793 10000000
3 Enter a, b, n: With n = 10000000 trapezoids, integral of f on [0, 3.14159] =
   1.999999999999809e+00
4 Execution time: 4.849014e-02 seconds
5
6 $ mpiexec -n 2 ./mpi_trap_generalized
7 0 3.141592653589793 10000000
8 Enter a, b, n: With n = 10000000 trapezoids, integral of f on [0, 3.14159] =
   1.99999999999962e+00
9 Execution time: 2.919581e-02 seconds
10
11 $ mpiexec -n 4 ./mpi_trap_generalized
12 0 3.141592653589793 10000000
13 Enter a, b, n: With n = 10000000 trapezoids, integral of f on [0, 3.14159] =
   2.000000000000020e+00
14 Execution time: 2.390280e-02 seconds
15
16 $ mpiexec --oversubscribe -n 8 ./mpi_trap_generalized
17 0 3.141592653589793 10000000
18 Enter a, b, n: With n = 10000000 trapezoids, integral of f on [0, 3.14159] =
   1.99999999999960e+00
19 Execution time: 1.339800e-02 seconds
```

Processos (p)	Tempo T_p (s)	Speedup S_p	Eficiência E_p
1	0.048490	1.00	1.00
2	0.029195	1.66	0.83
4	0.023902	2.03	0.51
8	0.013398	3.62	0.45

Tabela 1: Escalabilidade forte para a Regra do Trapézio ($n = 10^7$)

A eficiência por processo cai para menos de 0.8 a partir da execução com $p = 4$. Como a carga computacional por processo diminui ao aumentarmos p (escalabilidade forte), o tempo gasto com a passagem de mensagens torna-se desproporcionalmente grande. Nesse exemplo, o coordenador recebe mensagens sequencialmente ($p - 1$ chamadas de `MPI_Recv`), criando um gargalo linear.

(c) Bônus

Exercício 3 - Soma Paralela

Rodando a soma paralela com $n = 10000000$ (divisível por 1, 2, 4 e 8), temos:

```
1 $ mpiexec --oversubscribe -n 8 ./mpi_psum
2 Serial sum      = 4.999332562424394e+06
3 Parallel sum    = 4.999332562423944e+06
4 Relative error  = 8.997777161700452e-14
```

Realizando a comparação com um código que usa as *collectives*, vistas na aula 6, podemos ter uma redução de aproximadamente 20 linhas. Para essa contagem, fiz uma estimativa sem contar comentários, linhas livres, etc. De qualquer forma, contagens de linhas são, de certa forma, “subjetivas”.

De qualquer maneira, teríamos uma redução na complexidade do código, aproveitando algoritmos prontos da biblioteca.

Exercício 4 - Prática de Comunicação Coletiva

(a) Hello World com Collectives

A implementação das comunicações coletivas reduz a complexidade do código e otimiza a sincronização.

O uso de `MPI_Gather` substitui laços manuais de recebimento, centralizando as *greetings* no processo de rank = 0 e garantindo a exibição na ordem estrita dos ranks.

```
1 $ mpiexec --oversubscribe -n 8 ./hello_gather
2 Hello from rank 0/8 -- PID = 43747, CPU = 4
3 Hello from rank 1/8 -- PID = 43748, CPU = 13
4 Hello from rank 2/8 -- PID = 43749, CPU = 9
5 Hello from rank 3/8 -- PID = 43750, CPU = 12
6 Hello from rank 4/8 -- PID = 43751, CPU = 0
7 Hello from rank 5/8 -- PID = 43752, CPU = 7
8 Hello from rank 6/8 -- PID = 43753, CPU = 8
9 Hello from rank 7/8 -- PID = 43754, CPU = 2
```

(b) Minmax

```
1 $ mpiexec --oversubscribe -n 8 ./minmax
2 Max: Serial = 91.976658 | Parallel = 91.976658
3 Min: Serial = 4.784111 | Parallel = 4.784111
```

Aqui, o `MPI_Scatter` particiona o vetor para busca distribuída. Em seguida, o `MPI_Reduce` calcula os extremos usando as operações nativas `MPI_MAX` e `MPI_MIN`, gerando resultados alinhados à execução serial.

Exercício 5 - Soma Paralela de Vetores (Scatter / Gather)

O resultado do código inicial, disponibilizado no Classroom, é:

```
1 $ mpiexec --oversubscribe -n 8 ./mpi_vecadd_gather
2 z = x + y:
3   z[ 0] = 0
4   z[ 1] = 3
```

```
5  z[ 2] = 6
6  z[ 3] = 9
7  z[ 4] = 12
8  z[ 5] = 15
9  z[ 6] = 18
10 z[ 7] = 21
11 z[ 8] = 24
12 z[ 9] = 27
13 z[10] = 30
14 z[11] = 33
15 z[12] = 36
16 z[13] = 39
17 z[14] = 42
18 z[15] = 45
```

Estendendo o código para usar **AllGather**, observamos (naturalmente) o mesmo resultado.

Essa versão seria útil quando o vetor z é a entrada obrigatória para a próxima iteração do algoritmo em todos os nós. Por exemplo: se o próximo passo for calcular $w = M \times z$, todos os trabalhadores precisam conhecer z por completo.

Exercício 6 - Um tipo de dado derivado MPI

Primeiro, o resultado com um único broadcast:

```
1 $ mpiexec --oversubscribe -n 8 ./student_struct
2 Process 1 received: ID=10101, Name='Daniel Barros', Grade=9.80
3 Process 2 received: ID=10101, Name='Daniel Barros', Grade=9.80
4 Process 3 received: ID=10101, Name='Daniel Barros', Grade=9.80
5 Process 5 received: ID=10101, Name='Daniel Barros', Grade=9.80
6 Process 7 received: ID=10101, Name='Daniel Barros', Grade=9.80
7 Process 6 received: ID=10101, Name='Daniel Barros', Grade=9.80
8 Process 4 received: ID=10101, Name='Daniel Barros', Grade=9.80
```

Agora, o resultado da abordagem com 3 broadcasts:

```
1 $ mpiexec --oversubscribe -n 8 ./student_three_bcsts
2 Process 1 received: ID=10101, Name='Daniel Barros', Grade=9.80
3 Process 3 received: ID=10101, Name='Daniel Barros', Grade=9.80
4 Process 2 received: ID=10101, Name='Daniel Barros', Grade=9.80
5 Process 5 received: ID=10101, Name='Daniel Barros', Grade=9.80
6 Process 4 received: ID=10101, Name='Daniel Barros', Grade=9.80
7 Process 6 received: ID=10101, Name='Daniel Barros', Grade=9.80
8 Process 7 received: ID=10101, Name='Daniel Barros', Grade=9.80
```

A escolha entre as duas abordagens reflete o balanço entre trabalho de memória e custo de rede. O uso de um tipo derivado exige configurar previamente o layout da estrutura (usando `offsetof` para alinhar os dados e contornar o padding do compilador), mas permite transmitir tudo em apenas uma chamada de rede, pagando a penalidade de latência uma única vez.

Por outro lado, o envio por três chamadas separadas de `MPI_Bcast` simplifica o código ao dispensar essa configuração inicial, mas triplica a sobrecarga de comunicação, as barreiras de sincronização e a latência, tornando-se um possível gargalo de desempenho em aplicações reais.